
Evolving Neural Networks through Augmenting Topologies

Kenneth O. Stanley

kstanley@cs.utexas.edu

Department of Computer Sciences, The University of Texas at Austin, Austin, TX
78712, USA

Risto Miikkulainen

risto@cs.utexas.edu

Department of Computer Sciences, The University of Texas at Austin, Austin, TX
78712, USA

Abstract

An important question in neuroevolution is how to gain an advantage from evolving neural network topologies along with weights. We present a method, NeuroEvolution of Augmenting Topologies (NEAT), which outperforms the best fixed-topology method on a challenging benchmark reinforcement learning task. We claim that the increased efficiency is due to (1) employing a principled method of crossover of different topologies, (2) protecting structural innovation using speciation, and (3) incrementally growing from minimal structure. We test this claim through a series of ablation studies that demonstrate that each component is necessary to the system as a whole and to each other. What results is significantly faster learning. NEAT is also an important contribution to GAs because it shows how it is possible for evolution to both optimize *and complexify* solutions simultaneously, offering the possibility of evolving increasingly complex solutions over generations, and strengthening the analogy with biological evolution.

Keywords

Genetic algorithms, neural networks, neuroevolution, network topologies, speciation, competing conventions.

1 Introduction

Neuroevolution (NE), the artificial evolution of neural networks using genetic algorithms, has shown great promise in complex reinforcement learning tasks (Gomez and Miikkulainen, 1999; Gruau et al., 1996; Moriarty and Miikkulainen, 1997; Potter et al., 1995; Whitley et al., 1993). Neuroevolution searches through the space of behaviors for a network that performs well at a given task. This approach to solving complex control problems represents an alternative to statistical techniques that attempt to estimate the utility of particular actions in particular states of the world (Kaelbling et al., 1996). NE is a promising approach to solving reinforcement learning problems for several reasons. Past studies have shown NE to be faster and more efficient than reinforcement learning methods such as Adaptive Heuristic Critic and Q-Learning on single pole balancing and robot arm control (Moriarty and Miikkulainen, 1996; Moriarty, 1997). Because NE searches for a behavior instead of a value function, it is effective in problems with continuous and high-dimensional state spaces. In addition, memory is easily represented through recurrent connections in neural networks, making NE a natural choice for learning non-Markovian tasks (Gomez and Miikkulainen, 1999, 2002).

In traditional NE approaches, a topology is chosen for the evolving networks before the experiment begins. Usually, the network topology is a single hidden layer of neurons, with each hidden neuron connected to every network input and every network output. Evolution searches the space of connection weights of this fully-connected topology by allowing high-performing networks to reproduce. The weight space is explored through the crossover of network weight vectors and through the mutation of single networks' weights. Thus, the goal of fixed-topology NE is to optimize the connection weights that determine the functionality of a network.

However, connection weights are not the only aspect of neural networks that contribute to their behavior. The topology, or *structure*, of neural networks also affects their functionality. Modifying the network structure has been shown effective as part of supervised training (Chen et al., 1993). There has also been a great deal of interest in evolving network topologies as well as weights over the last decade (Angeline et al., 1993; Branke, 1995; Gruau et al., 1996; Yao, 1999). The basic question, however, remains: Can evolving topologies along with weights provide an advantage over evolving weights on a fixed-topology? A fully connected network can in principle approximate any continuous function (Cybenko, 1989). So why waste valuable effort permuting over different topologies?

The answers provided thus far are inconclusive. Some have argued that network complexity can affect the speed and accuracy of learning (Zhang and Mühlenbein, 1993). Although this assertion is true for the backpropagation algorithm, it is not clear whether it applies when weights are being optimized by evolution and *not* backpropagation.

A persuasive argument for the evolution of both topology and weights was put forward by Gruau et al. (1996), who claimed that evolving structure saves the time wasted by humans trying to decide on the topology of networks for a particular NE problem. Although almost all fixed-topology NE systems use a fully connected hidden layer, deciding how many hidden nodes are needed is a trial-and-error process. Gruau et al. supported their argument by evolving the topology and weights of an artificial neural network that solved the hardest pole-balancing benchmark problem to date. However, later results suggested that structure was not necessary to solve the difficult problem. A fixed-topology method called *Enforced Subpopulations* (ESP) (Gomez and Miikkulainen, 1999) was able to solve the same problem 5 times faster simply by restarting with a random number of hidden neurons whenever it became stuck.

This article aims to demonstrate the opposite conclusion: if done right, evolving structure along with connection weights can significantly enhance the performance of NE. We present a novel NE method called *NeuroEvolution of Augmenting Topologies* (NEAT) that is designed to take advantage of structure as a way of minimizing the dimensionality of the search space of connection weights. If structure is evolved such that topologies are minimized and grown incrementally, significant gains in learning speed result. Improved efficiency results from topologies being minimized *throughout* evolution, rather than only at the very end.

Evolving structure incrementally presents several technical challenges: (1) Is there a genetic representation that allows disparate topologies to cross over in a meaningful way? (2) How can topological innovation that needs a few generations to be optimized be protected so that it does not disappear from the population prematurely? (3) How can topologies be minimized *throughout evolution* without the need for a specially contrived fitness function that measures complexity?

The NEAT method consists of solutions to each of these problems as will be described below. The method is validated on pole balancing tasks, where NEAT performs 25 times faster than Cellular Encoding and 5 times faster than ESP. The results show that structure is a powerful resource in NE when appropriately utilized. NEAT is unique because structures become increasingly more complex as they become more optimal, strengthening the analogy between GAs and natural evolution.

2 Background

Many systems have been developed over the last decade that evolve both neural network topologies and weights (Angeline et al., 1993; Braun and Weisbrod, 1993; Dasgupta and McGregor, 1992; Fullmer and Miikkulainen, 1992; Gruau et al., 1996; Krishnan and Ciesielski, 1994; Lee and Kim, 1996; Mandischer, 1993; Maniezzo, 1994; Opitz and Shavlik, 1997; Pujol and Poli, 1998; Yao and Liu, 1996; Zhang and Mühlenbein, 1993). These methods encompass a range of ideas about how *Topology and Weight Evolving Artificial Neural Networks* (TWEANNs) should be implemented. In this section, we address some of the ideas and assumptions about the design of TWEANNs, and offer solutions to some unsolved problems. Our goal is to find how a neuroevolution method can use the evolution of topology to increase its efficiency.

2.1 TWEANN Encoding

The question of how to encode networks using an efficient genetic representation must be addressed by all TWEANNs. We will discuss several prototypical representational schemes.

TWEANNs can be divided between those that use a direct encoding, and those that use an indirect one. Direct encoding schemes, employed by most TWEANNs, specify in the genome every connection and node that will appear in the phenotype (Angeline et al., 1993; Braun and Weisbrod, 1993; Dasgupta and McGregor, 1992; Fullmer and Miikkulainen, 1992; Krishnan and Ciesielski, 1994; Lee and Kim, 1996; Maniezzo, 1994; Opitz and Shavlik, 1997; Pujol and Poli, 1998; Yao and Liu, 1996; Zhang and Mühlenbein, 1993). In contrast, indirect encodings usually only specify rules for constructing a phenotype (Gruau, 1993; Mandischer, 1993). These rules can be layer specifications or growth rules through cell division. Indirect encoding allows a more compact representation than direct encoding, because every connection and node are not specified in the genome, although they can be derived from it.

2.1.1 Binary Encoding

Direct encodings usually require simpler implementations than indirect encodings. The simplest implementation is based on the traditional bit string representation used by GAs. For example, Dasgupta and McGregor (1992) use such an encoding in their method, called *Structured Genetic Algorithm* (sGA), where a bit string represents the connection matrix of a network. sGA is notable for its simplicity, allowing it to operate almost like a standard GA. However, there are several limitations as well. First, the size of the connectivity matrix is the square of the number of nodes. Thus, the representation blows up for a large number of nodes. Second, because the size of the bit string must be the same for all organisms, the maximum number of nodes (and hence connections as well) must be chosen by a human running the system, and if the maximum is not sufficient, the experiment must be repeated. Third, using a linear string of bits to represent a graph structure makes it difficult to ensure that crossover will yield useful combinations.

2.1.2 Graph Encoding

Because bit strings are not the most natural representation for networks, most TWEANNs use encodings that represent graph structures more explicitly. Pujol and Poli (1997) use a dual representation scheme to allow different kinds of crossover in their *Parallel Distributed Genetic Programming* (PDGP) system. The first representation is a graph structure. The second is a linear genome of node definitions specifying incoming and outgoing connections. The idea is that different representations are appropriate for different kinds of operators. Subgraph-swapping crossovers and topological mutations use the grid, while point crossovers and connection parameter mutations use the linear representation.

As in sGA, PDGP has a finite limit on the number of nodes in the network, corresponding to the number of nodes in the two-dimensional grid that represents the graph version of the genome. PDGP uses graph encoding so that subgraphs can be swapped in crossover. Subgraph swapping is representative of a prevailing philosophy in TWEANNs that subgraphs are functional units and therefore swapping them makes sense because it preserves the structure of functional components. However, we cannot be sure whether the particular subgraphs being combined in PDGP are the right ones to create a functional offspring.

2.1.3 Nonmating

Because crossover of networks with different topologies can frequently lead to a loss of functionality, some researchers have given up on crossover altogether in what is called Evolutionary Programming (Yao and Liu, 1996). Angeline et al. (1993) implemented a system called *Generalized Acquisition of Recurrent Links* (GNARL), commenting that “the prospect of evolving connectionist networks with crossover appears limited in general.” Although GNARL uses a graph encoding, it is fundamentally different from PDGP in that it sidesteps the issue of crossover entirely. GNARL demonstrates that a TWEANN does not need crossover to work, leaving the problem of demonstrating the advantages of crossover to other methods.

2.1.4 Indirect Encoding

Gruau’s (1993) *Cellular Encoding* (CE) method is an example of a system that utilizes indirect encoding of network structures. In CE, genomes are programs written in a specialized graph transformation language. The transformations are motivated by nature in that they specify *cell divisions*. Different kinds of connectivities can result from a division, so there are several kinds of cell divisions possible. A major advantage of CE is that its genetic representations are compact. Genes in CE can be reused multiple times during the development of a network, each time requesting a cell division at a different location. CE shows that cell divisions can encode the development of networks from a single cell, much as organisms in nature begin as a single cell that differentiates as it splits into more cells.

Although CE demonstrates that it is possible to evolve developmental systems, we chose direct encoding for NEAT because, as Braun and Weisbrod (1993) argue, indirect encoding requires “more detailed knowledge of genetic and neural mechanisms.” In other words, because indirect encodings do not map directly to their phenotypes, they can bias the search in unpredictable ways. To make good use of indirect encodings, we need to first understand them well enough to make sure that they do not focus the search on some suboptimal class of topologies. Further, experimental results suggest that CE is not necessarily more efficient than direct encoding methods (Section 4.3.3).

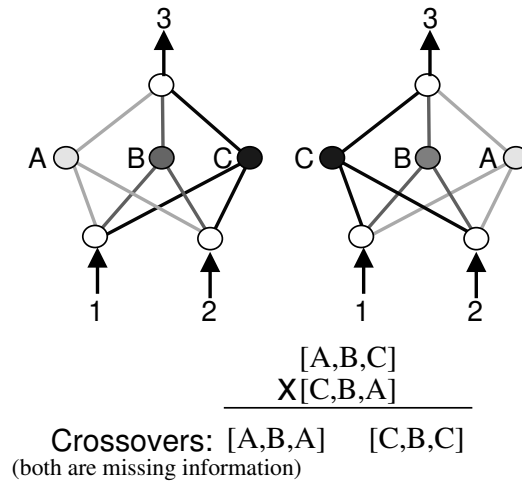


Figure 1: The competing conventions problem. The two networks compute the same exact function even though their hidden units appear in a different order and are represented by different chromosomes, making them incompatible for crossover. The figure shows that the two single-point recombinations are both missing one of the 3 main components of each solution. The depicted networks are only 2 of the 6 possible permutations of hidden unit orderings.

We now turn to several specific problems with TWEANNs and address each in turn.

2.2 Competing Conventions

One of the main problems for NE is the *Competing Conventions Problem* (Montana and Davis, 1989; Schaffer et al., 1992), also known as the *Permutations Problem* (Radcliffe, 1993). Competing conventions means having more than one way to express a solution to a weight optimization problem with a neural network. When genomes representing the same solution do not have the same encoding, crossover is likely to produce damaged offspring.

Figure 1 depicts the problem for a simple 3-hidden-unit network. The three hidden neurons A , B , and C , can represent the same general solution in $3! = 6$ different permutations. When one of these permutations crosses over with another, critical information is likely to be lost. For example, crossing $[A, B, C]$ and $[C, B, A]$ can result in $[C, B, C]$, a representation that has lost one third of the information that both of the parents had. In general, for n hidden units, there are $n!$ functionally equivalent solutions. The problem can be further complicated with *differing* conventions, i.e., $[A, B, C]$ and $[D, B, E]$, which share functional interdependence on B .

An even more difficult form of competing conventions is present in TWEANNs, because TWEANN networks can represent similar solutions using entirely different topologies, or even genomes of different sizes. Because TWEANNs do not satisfy strict constraints on the kinds of topologies they produce, proposed solutions to the competing conventions problem for fixed or constrained topology networks such as nonredundant genetic encoding (Thierens, 1996) do not apply. Radcliffe (1993) goes as far as calling an integrated scheme combining connectivity and weights the “Holy Grail in

this area.” Although some TWEANNs such as PDGP have attempted to address the problem by assuming that subnetworks represent functional units that can be recombined, different topologies may not be based on the same subnetworks at all, in which case no meaningful combination of substructures exists.

The main intuition behind NEAT originates from the fundamental problem with representing different structures: their representations will not necessarily match up. Sometimes, the genomes can have different sizes. Other times, genes in the exact same position on different chromosomes may be expressing completely different traits. In addition, genes expressing the same trait may appear at different positions on different chromosomes. How can these complications be resolved?

Nature faces a similar problem with gene alignment in sexual reproduction. Genomes in nature are not of fixed-length either. Somewhere along the evolution from single cells to more complex organisms, new genes were added to the genomes in a process called *gene amplification* (Darnell and Doolittle, 1986; Watson et al., 1987). If new genes could just randomly insert themselves in positions on the genome without any indication of which gene is which, life probably would not have succeeded, because the competing conventions problem would decimate a huge chunk of offspring. There needed to be some way to keep crossover orderly, so that the *right* genes could be crossed with the right genes.

Nature’s solution utilizes *homology*: two genes are homologous if they are alleles of the same trait. For example, in *E. coli*, in a process called synapsis, a special protein called RecA goes through and lines up homologous genes between two genomes before crossover occurs (Radding, 1982; Sigal and Alberts, 1972). Actual homology between neural networks cannot be easily ascertained by direct structural analysis (hence, the competing conventions problem). The main insight in NEAT is that the *historical origin* of two genes is direct evidence of homology if the genes share the same origin. Thus, NEAT performs *artificial synapsis* based on historical markings, allowing it to add new structure without losing track of which gene is which over the course of a simulation.

2.3 Protecting Innovation with Speciation

In TWEANNs, innovation takes place by adding new structure to networks through mutation. Frequently, adding new structure initially causes the fitness of a network to decrease. For example, adding a new node introduces a nonlinearity where there was none before; adding a new connection can reduce fitness before its weight has a chance to optimize. It is unlikely that a new node or connection just happens to express a useful function as soon as it is introduced. Some generations are required to optimize the new structure and make use of it. Unfortunately, because of the initial loss of fitness caused by the new structure, the innovation is unlikely to survive in the population long enough to be optimized. Thus, it is necessary to somehow protect networks with structural innovations so they have a chance to make use of their new structure.

The GNARL system addresses the problem of protecting innovation by adding nonfunctional structure. A node is added to a genome without any connections, in the hopes that in the future some useful connections will develop. However, nonfunctional structures may never end up connecting to the functional network, adding extraneous parameters to the search.

In nature, different structures tend to be in different species that compete in different niches. Thus, innovation is implicitly protected within a niche. Similarly, if networks with innovative structures could be isolated into their own species, they would

have a chance to optimize their structures before having to compete with the population at large.

Speciation, also known as *nicheing*, has been studied in GAs, but is not usually applied to neuroevolution. Speciation is most commonly applied to multimodal function optimization (Mahfoud, 1995), where a function has multiple optima, and a GA with several species is used to find those optima. Speciation has also been applied in the cooperative coevolution of modular systems of multiple solutions (Darwen and Yao, 1996; Potter and De Jong, 1995).

Speciation requires a compatibility function to tell whether two genomes should be in the same species or not. It is difficult to formulate such a compatibility function between networks of different topologies, which may be the reason why speciation has not been brought into TWEANNs. The competing conventions problem makes measuring compatibility particularly problematic because networks that compute the same function can appear very different.

However, because NEAT has a solution to the competing conventions problem, using historical information about genes, the population in NEAT can easily be speciated. We use *explicit fitness sharing*, which forces individuals with similar genomes to share their fitness payoff (Goldberg and Richardson, 1987). The original implicit version of fitness sharing introduced by Holland (1975) grouped individuals by performance similarity rather than genetic similarity. The explicit version is appropriate for TWEANNs because it allows grouping networks based on topology and weight configurations. The result of sharing fitness is that the number of networks that can exist in the population on a single fitness peak is limited by the size of the peak. Therefore, the population divides into a number of species, each on a different peak, without the threat of any one species taking over. Explicit fitness sharing is well-suited for NEAT, because similarity can easily be measured based on the historical information in the genes. Thus, innovations in NEAT are protected in their own species.

2.4 Initial Populations and Topological Innovation

In many TWEANN systems, the initial population is a collection of random topologies. Such a population ensures topological diversity from the start. However, random initial populations turn out to produce many problems for TWEANNs. For example, under many of the direct encoding schemes, there is a chance that a network will have no path from each of its inputs to its outputs. Such infeasible networks take time to weed out of the population.

However, there is a more subtle and more serious problem with starting randomly. As our experiments will confirm (Section 5.4), it is desirable to evolve minimal solutions; that way, the number of parameters that have to be searched is reduced. Starting out with random topologies does not lead to finding minimal solutions, since the population starts out with many unnecessary nodes and connections already present. None of these nodes or connections have had to withstand a single evaluation, meaning there is no justification for their configuration. Any minimization of networks would have to be spent getting rid of apparatus that should not have been there in the first place, and nothing in the process of recombining different topologies pushes towards such minimization. Since there is no fitness cost in creating larger networks, they will dominate as long as they have high fitness.

One way to force minimal topologies is to incorporate network size into the fitness function, and some TWEANNs actually do this (Zhang and Mühlenbein, 1993). In such methods, larger networks have their fitnesses penalized. Although altering the

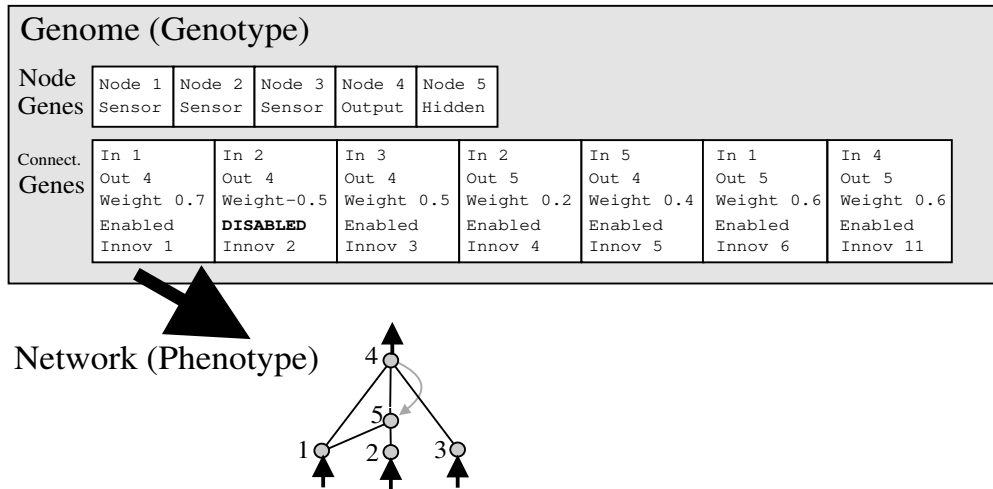


Figure 2: A genotype to phenotype mapping example. A genotype is depicted that produces the shown phenotype. There are 3 input nodes, one hidden, and one output node, and seven connection definitions, one of which is recurrent. The second gene is disabled, so the connection that it specifies (between nodes 2 and 4) is not expressed in the phenotype.

fitness function in this way can encourage smaller networks, it is difficult to know how large the penalty should be for any particular network size, particularly because different problems may have significantly different topological requirements. Altering the fitness function is ad hoc and may cause evolution to perform differently than the designer of the original unmodified fitness function intended.

An alternative solution is for the neuroevolution method itself to tend towards minimality. If the population begins with no hidden nodes and grows structure only as it benefits the solution, there is no need for ad hoc fitness modification to minimize networks. Therefore, starting out with a minimal population and growing structure from there is a design principle in NEAT.

By starting out minimally, NEAT ensures that the system searches for the solution in the lowest-dimensional weight space possible over the course of *all* generations. Thus, the goal is not to minimize only the final product, but all *intermediate* networks along the way as well. This idea is key to gaining an advantage from the evolution of topology: it allows us to minimize the search space, resulting in dramatic performance gains. One reason current TWEANNS do not start out minimally is that without topological diversity present in the initial population, topological innovations would not survive. The problem of protecting innovation is not addressed by these methods, so networks with major structural additions are likely not to reproduce. Thus, speciating the population enables starting minimally in NEAT.

3 NeuroEvolution of Augmenting Topologies (NEAT)

The NEAT method, as described in detail in this section, consists of putting together the ideas developed in the previous section into one system. We begin by explaining

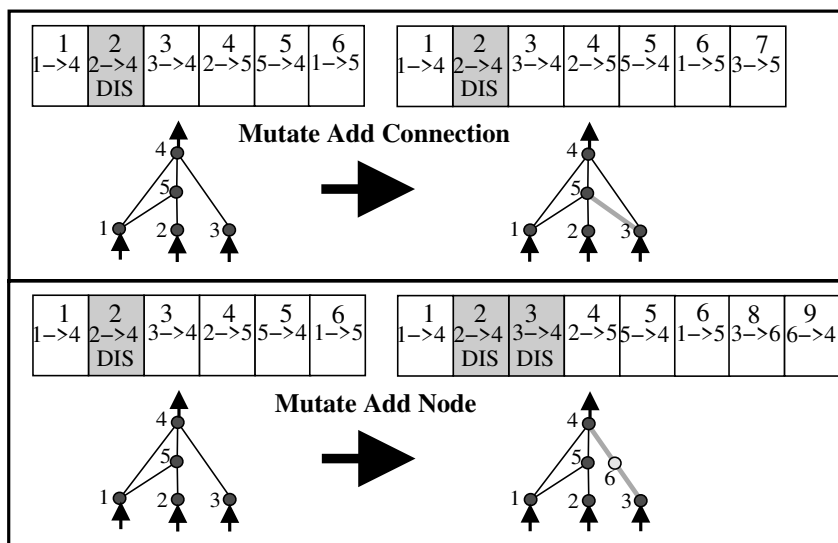


Figure 3: The two types of structural mutation in NEAT. Both types, adding a connection and adding a node, are illustrated with the connection genes of a network shown above their phenotypes. The top number in each genome is the *innovation number* of that gene. The innovation numbers are historical markers that identify the original historical ancestor of each gene. New genes are assigned new increasingly higher numbers. In adding a connection, a single new connection gene is added to the end of the genome and given the next available innovation number. In adding a new node, the connection gene being split is disabled, and two new connection genes are added to the end the genome. The new node is between the two new connections. A new node gene (not depicted) representing this new node is added to the genome as well.

the genetic encoding used in NEAT and continue by describing the components that specifically address each of the three problems of TWEANNs.

3.1 Genetic Encoding

NEAT's genetic encoding scheme is designed to allow corresponding genes to be easily lined up when two genomes cross over during mating. Genomes are linear representations of network connectivity (Figure 2). Each genome includes a list of *connection genes*, each of which refers to two *node genes* being connected. Node genes provide a list of inputs, hidden nodes, and outputs that can be connected. Each connection gene specifies the in-node, the out-node, the weight of the connection, whether or not the connection gene is expressed (an enable bit), and an *innovation number*, which allows finding corresponding genes (as will be explained below).

Mutation in NEAT can change both connection weights and network structures. Connection weights mutate as in any NE system, with each connection either perturbed or not at each generation. Structural mutations occur in two ways (Figure 3). Each mutation expands the size of the genome by adding gene(s). In the *add connection* mutation, a single new connection gene with a random weight is added connecting two previously unconnected nodes. In the *add node* mutation, an existing connection is

split and the new node placed where the old connection used to be. The old connection is disabled and two new connections are added to the genome. The new connection leading into the new node receives a weight of 1, and the new connection leading out receives the same weight as the old connection. This method of adding nodes was chosen in order to minimize the initial effect of the mutation. The new nonlinearity in the connection changes the function slightly, but new nodes can be immediately integrated into the network, as opposed to adding extraneous structure that would have to be evolved into the network later. This way, because of speciation, the network will have time to optimize and make use of its new structure.

Through mutation, the genomes in NEAT will gradually get larger. Genomes of varying sizes will result, sometimes with different connections at the same positions. The most complex form of the competing conventions problem, with numerous differing topologies and weight combinations, is an inevitable result of allowing genomes to grow unbounded. How can NE cross over differently sized genomes in a sensible way? The next section explains how NEAT addresses this problem.

3.2 Tracking Genes through Historical Markings

There is unexploited information in evolution that tells us exactly which genes match up with which genes between *any* individuals in a topologically diverse population. That information is the historical origin of each gene. Two genes with the same historical origin must represent the same structure (although possibly with different weights), since they are both derived from the same ancestral gene of some point in the past. Thus, all a system needs to do to know which genes line up with which is to keep track of the historical origin of every gene in the system.

Tracking the historical origins requires very little computation. Whenever a new gene appears (through structural mutation), a *global innovation number* is incremented and assigned to that gene. The innovation numbers thus represent a chronology of the appearance of every gene in the system. As an example, let us say the two mutations in Figure 3 occurred one after another in the system. The new connection gene created in the first mutation is assigned the number 7, and the two new connection genes added during the new node mutation are assigned the numbers 8 and 9. In the future, whenever these genomes mate, the offspring will inherit the same innovation numbers on each gene; innovation numbers are never changed. Thus, the historical origin of every gene in the system is known throughout evolution.

A possible problem is that the same structural innovation will receive different innovation numbers in the same generation if it occurs by chance more than once. However, by keeping a list of the innovations that occurred in the current generation, it is possible to ensure that when the same structure arises more than once through independent mutations in the same generation, each identical mutation is assigned the same innovation number. Thus, there is no resultant explosion of innovation numbers.

The historical markings give NEAT a powerful new capability. The system now knows exactly which genes match up with which (Figure 4). When crossing over, the genes in both genomes with the same innovation numbers are lined up. These genes are called *matching* genes. Genes that do not match are either *disjoint* or *excess*, depending on whether they occur within or outside the range of the other parent's innovation numbers. They represent structure that is not present in the other genome. In composing the offspring, genes are randomly chosen from either parent at matching genes, whereas all excess or disjoint genes are always included from the more fit parent. This way, historical markings allow NEAT to perform crossover using linear genomes with-

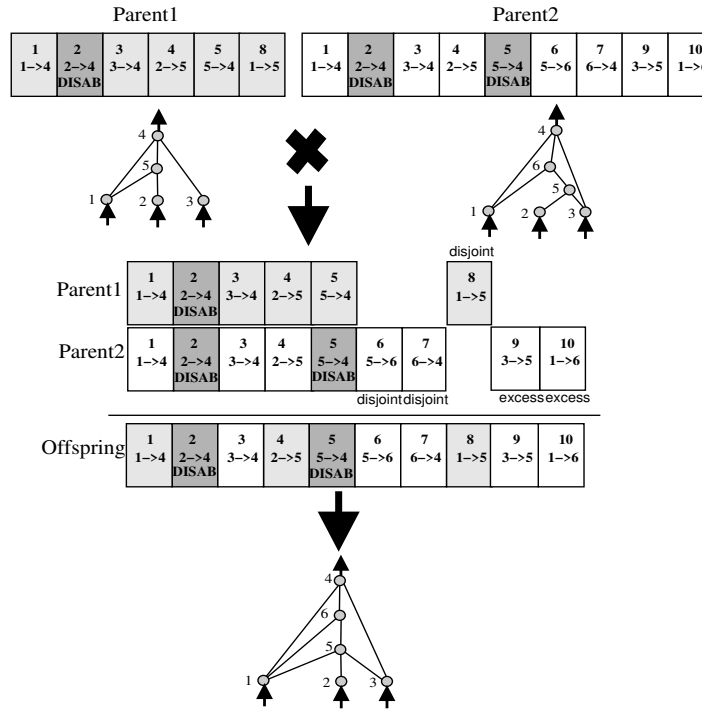


Figure 4: Matching up genomes for different network topologies using innovation numbers. Although Parent 1 and Parent 2 look different, their innovation numbers (shown at the top of each gene) tell us which genes match up with which. Even without any topological analysis, a new structure that combines the overlapping parts of the two parents as well as their different parts can be created. Matching genes are inherited randomly, whereas disjoint genes (those that do not match in the middle) and excess genes (those that do not match in the end) are inherited from the more fit parent. In this case, equal fitnesses are assumed, so the disjoint and excess genes are also inherited randomly. The disabled genes may become enabled again in future generations: there's a preset chance that an inherited gene is disabled if it is disabled in either parent.

out the need for expensive topological analysis.

By adding new genes to the population and sensibly mating genomes representing different structures, the system can form a population of diverse topologies. However, it turns out that such a population on its own cannot maintain topological innovations. Because smaller structures optimize faster than larger structures, and adding nodes and connections usually initially decreases the fitness of the network, recently augmented structures have little hope of surviving more than one generation even though the innovations they represent might be crucial towards solving the task in the long run. The solution is to protect innovation by speciating the population, as explained in the next section.

3.3 Protecting Innovation through Speciation

Speciating the population allows organisms to compete primarily within their own niches instead of with the population at large. This way, topological innovations are

protected in a new niche where they have time to optimize their structure through competition within the niche. The idea is to divide the population into species such that similar topologies are in the same species. This task appears to be a topology matching problem. However, it again turns out that historical markings offer an efficient solution.

The number of excess and disjoint genes between a pair of genomes is a natural measure of their compatibility distance. The more disjoint two genomes are, the less evolutionary history they share, and thus the less compatible they are. Therefore, we can measure the compatibility distance δ of different structures in NEAT as a simple linear combination of the number of excess E and disjoint D genes, as well as the average weight differences of matching genes $\overline{W}$, including disabled genes:

$$\delta = \frac{c_1 E}{N} + \frac{c_2 D}{N} + c_3 \cdot \overline{W}. \quad (1)$$

The coefficients c_1 , c_2 , and c_3 allow us to adjust the importance of the three factors, and the factor N , the number of genes in the larger genome, normalizes for genome size (N can be set to 1 if both genomes are small, i.e., consist of fewer than 20 genes).

The distance measure δ allows us to speciate using a compatibility threshold δ_t . An ordered list of species is maintained. In each generation, genomes are sequentially placed into species. Each existing species is represented by a random genome inside the species from the *previous generation*. A given genome g in the current generation is placed in the first species in which g is compatible with the representative genome of that species. This way, species do not overlap.¹ If g is not compatible with any existing species, a new species is created with g as its representative.

As the reproduction mechanism for NEAT, we use *explicit fitness sharing* (Goldberg and Richardson, 1987), where organisms in the same species must share the fitness of their niche. Thus, a species cannot afford to become too big even if many of its organisms perform well. Therefore, any one species is unlikely to take over the entire population, which is crucial for speciated evolution to work. The adjusted fitness f'_i for organism i is calculated according to its distance δ from every other organism j in the population:

$$f'_i = \frac{f_i}{\sum_{j=1}^n \text{sh}(\delta(i, j))}. \quad (2)$$

The sharing function sh is set to 0 when distance $\delta(i, j)$ is above the threshold δ_t ; otherwise, $\text{sh}(\delta(i, j))$ is set to 1 (Spears, 1995). Thus, $\sum_{j=1}^n \text{sh}(\delta(i, j))$ reduces to the number of organisms in the same species as organism i . This reduction is natural since species are already clustered by compatibility using the threshold δ_t . Every species is assigned a potentially different number of offspring in proportion to the sum of adjusted fitnesses f'_i of its member organisms. Species then reproduce by first eliminating the lowest performing members from the population. The entire population is then replaced by the offspring of the remaining organisms in each species.²

The net desired effect of speciating the population is to protect topological innovation. The final goal of the system, then, is to perform the search for a solution as efficiently as possible. This goal is achieved through minimizing the dimensionality of the search space.

¹It is also possible to determine the compatibility of a genome g with a species s by using the average compatibility of g with every genome in a species s , but in practice, only comparing to the first genome in s is sufficient and takes constant time.

²In rare cases when the fitness of the entire population does not improve for more than 20 generations, only the top two species are allowed to reproduce, refocusing the search into the most promising spaces.

3.4 Minimizing Dimensionality through Incremental Growth from Minimal Structure

As discussed in Section 2.4, TWEANNs typically start with an initial population of random topologies in order to introduce diversity from the outset. In contrast, NEAT biases the search towards minimal-dimensional spaces by starting out with a *uniform* population of networks with zero hidden nodes (i.e., all inputs connect directly to outputs). New structure is introduced incrementally as structural mutations occur, and only those structures survive that are found to be useful through fitness evaluations. In other words, the structural elaborations that occur in NEAT are always justified. Since the population starts minimally, the dimensionality of the search space is minimized, and NEAT is always searching through fewer dimensions than other TWEANNs and fixed-topology NE systems. Minimizing dimensionality gives NEAT a performance advantage compared to other approaches, as will be discussed next.

4 Performance Evaluations

We evaluate the system's performance in order to answer two questions: (1) Can NEAT evolve the necessary structures? (2) Can NEAT find solutions more efficiently than other neuroevolution systems? The first question establishes that topology building indeed happens in NEAT in a reliable way, meaning that NEAT will grow new structure to cope with problems that require it. For this reason, NEAT is applied to the problem of building an XOR network. Although this task is simple, it requires growing hidden units, and therefore serves as a simple test for the method.

The second question is answered in the course of successively more difficult pole balancing tasks, where the objective is to balance two poles attached to a cart by moving the cart in appropriate directions to keep the pole from falling. Pole balancing is a good benchmark task because there are many different systems available for comparison. The most difficult problem of balancing two poles without velocity information, a non-Markovian task, provides very strong evidence that evolving augmenting topologies is not only interesting for its capacity to find structures, but is also efficient in difficult control tasks.

4.1 Parameter Settings

The same experimental settings are used in all experiments; they were not tuned specifically for any particular problem. The one exception is the hardest pole balancing problem (Double pole, no velocities, or DPNV) where a larger population size was used to match those of other systems in this task. Because some of NEAT's system parameters are sensitive to population size, we altered them accordingly.

All experiments except DPNV, which had a population of 1,000, used a population of 150 NEAT networks. The coefficients for measuring compatibility were $c_1 = 1.0$, $c_2 = 1.0$, and $c_3 = 0.4$. With DPNV, c_3 was increased to 3.0 in order to allow for finer distinctions between species based on weight differences (the larger population has room for more species). In all experiments, $\delta_t = 3.0$, except in DPNV where it was 4.0, to make room for the larger weight significance coefficient c_3 . If the maximum fitness of a species did not improve in 15 generations, the networks in the stagnant species were not allowed to reproduce. The champion of each species with more than five networks was copied into the next generation unchanged. There was an 80% chance of a genome having its connection weights mutated, in which case each weight had a 90% chance of being uniformly perturbed and a 10% chance of being assigned a new random value. (The system is tolerant to frequent mutations because of the protection speciation pro-

vides.) There was a 75% chance that an inherited gene was disabled if it was disabled in either parent. In each generation, 25% of offspring resulted from mutation without crossover. The interspecies mating rate was 0.001. In smaller populations, the probability of adding a new node was 0.03 and the probability of a new link mutation was 0.05. In the larger population, the probability of adding a new link was 0.3, because a larger population can tolerate a larger number of prospective species and greater topological diversity. We used a modified sigmoidal transfer function, $\varphi(x) = \frac{1}{1+e^{-4.9x}}$, at all nodes. The steepened sigmoid allows more fine tuning at extreme activations. It is optimized to be close to linear during its steepest ascent between activations -0.5 and 0.5 . These parameter values were found experimentally: links need to be added significantly more often than nodes, and an average weight difference of 3.0 is about as significant as one disjoint or excess gene. Performance is robust to moderate variations in these values.

4.2 Verification: Evolving XORs

Because XOR is not linearly separable, a neural network requires hidden units to solve it. The two inputs must be combined at some hidden unit, as opposed to only at the output node, because there is no function over a linear combination of the inputs that can separate the inputs into the proper classes. These structural requirements make XOR suitable for testing NEAT's ability to evolve structure. For example, NEAT's method for adding new nodes might be too destructive to allow new nodes to get into the population. Or, it could find a local champion with a wrong kind of connectivity that dominates the population so much that the systems fails to evolve the proper connectivity. Third, maybe the changing structure renders past connection weight values obsolete. If so, the algorithm would have trouble enlarging topologies that are already largely specialized. This experiment is meant to show that NEAT is not impeded by such potential obstacles, but can grow structure efficiently and consistently when needed.

To compute fitness, the distance of the output from the correct answer was summed for all four input patterns. The result of this error was subtracted from 4 so that higher fitness would reflect better network structure. The resulting number was squared to give proportionally more fitness the closer a network was to a solution.

The initial generation consisted of networks with no hidden units (Figure 5(a)). The networks had 2 inputs, 1 bias unit, and 1 output. The bias unit is an input that is always set to 1.0. There were three connection genes in each genome in the initial population. Two genes connected the inputs to the output, and one connected the bias to the output. Each connection gene received a random connection weight.

On 100 runs, the first experiment shows that the NEAT system finds a structure for XOR in an average of 32 generations (4,755 networks evaluated, std 2,553). On average a solution network had 2.35 hidden nodes and 7.48 nondisabled connection genes. The number of nodes and connections is close to optimal considering that the smallest possible network has a single hidden unit (Figure 5(b)). NEAT is very consistent in finding a solution. It did not fail once in 100 simulations. The worst performance took 13,459 evaluations, or about 90 generations (compared to 32 generations on average). The standard deviation for number of nodes used in a solution was 1.11, meaning NEAT very consistently used 1 or 2 hidden nodes to build an XOR network. In conclusion, NEAT solves the XOR problem without trouble and in doing so keeps the topology small.

The XOR problem has been used to demonstrate performance of several prior TWEANN algorithms. Unfortunately, quantitative performance comparisons are dif-

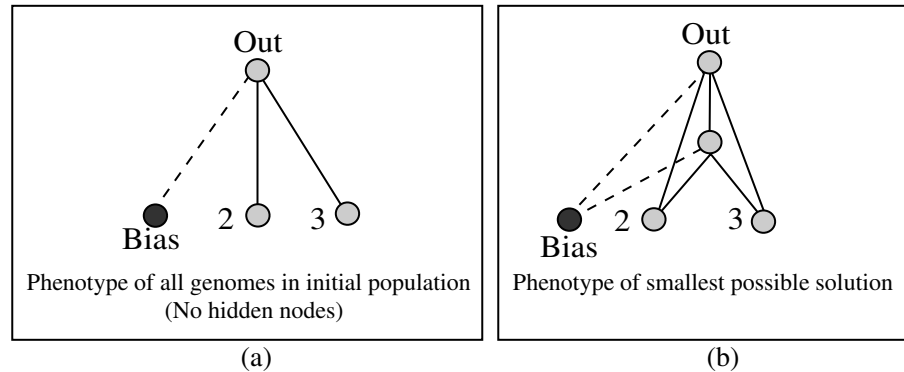


Figure 5: Initial phenotype and optimal XOR. Figure (a) shows the phenotype given to the entire initial population. Notice that there are no hidden nodes. In NEAT, a bias is a node that can connect to any node other than inputs. Figure (b) shows an optimal solution with only 1 hidden node. (A network without hidden nodes cannot compute XOR.) The bias connections are not always needed depending on the solution; All other connections are necessary. The optimal (1 hidden node) solution was found in 22 of 100 runs. The average solution had 2.35 hidden nodes with a standard deviation of 1.11 nodes.

difficult in this domain because the methodologies vary widely across experiments (Dasgupta and McGregor, 1992; Pujol and Poli, 1998; Yao and Shi, 1995; Zhang and Mühlenbein, 1993). Also, XOR is simple and artificial and does not challenge modern methods in the way real-world problems do. Let us next turn to benchmark problems where more substantial performance comparisons are possible.

4.3 Pole Balancing as a Benchmark Task

There are many control learning tasks where the techniques employed in NEAT can make a difference. Many of these potential applications, like robot navigation or game playing, present problems without known solutions. We use the pole balancing domain for comparison because it is a known benchmark in the literature, which makes it possible to demonstrate the effectiveness of NEAT compared to others. It is also a good surrogate for real problems, in part because pole balancing in fact *is* a real task, and also because the difficulty can be adjusted. Earlier comparisons were done with a single pole (Moriarty and Miikkulainen, 1996), but this version of the task has become too easy for modern methods. Balancing two poles simultaneously is on the other hand challenging enough for all current methods.

Therefore, we demonstrate the advantage of evolving structure through double pole balancing experiments. Two poles are connected to a moving cart by a hinge and the neural network must apply force to the cart to keep the poles balanced for as long as possible without going beyond the boundaries of the track. The system state is defined by the cart position x and velocity $\dot{x}$, the first pole's position θ_1 and angular velocity $\dot{\theta}_1$, and the second pole's position θ_2 and angular velocity $\dot{\theta}_2$. Control is possible because the poles have different lengths and therefore respond differently to control inputs.

Standard reinforcement learning methods have also been applied to this task (Anderson, 1989; Pendrith, 1994). However, we limit the comparisons in this paper to

NE methods for two reasons: (1) The focus is on developing and demonstrating better performance on evolving neural networks and (2) NE methods in this comparison have outperformed reinforcement learning methods in prior comparisons on the pole balancing task (Moriarty and Miikkulainen, 1996). Thus, the question here is whether evolving structure can lead to greater NE performance.

4.3.1 Pole Balancing Comparisons

We set up the pole balancing experiments as described by Wieland (1991) and Gomez and Miikkulainen (1999). The Runge-Kutta fourth-order method was used to implement the dynamics of the system, with a step size of 0.01s. All state variables were scaled to $[-1.0, 1.0]$ before being fed to the network. Networks output a force every 0.02 seconds between $[-10, 10]N$. The poles were 0.1m and 1.0m long. The initial position of the long pole was 1° and the short pole was upright; the track was 4.8 meters long.

Two versions of the double pole balancing task are used: one with velocity inputs included and another without velocity information. The first task is Markovian and allows comparing to many different systems. Taking away velocity information makes the task more difficult because the network must estimate an internal state in lieu of velocity, which requires recurrent connections.

On the double pole balancing with velocity (DPV) problem, NEAT is compared to published results from four other NE systems. The first two represent standard population-based approaches. Saravanan and Fogel (1995) used Evolutionary Programming, which relies entirely on mutation of connection weights, while Wieland (1991) used both mating and mutation. The second two systems, SANE (Moriarty and Miikkulainen, 1996) and ESP (Gomez and Miikkulainen, 1999), evolved populations of neurons and a population of network blueprints that specifies how to build networks from the neurons that are assembled into fixed-topology networks for evaluation. The topologies are fixed because the individual neurons are always placed into pre-designated slots in the neural networks they compose. SANE maintains a single population of neurons. ESP improves over SANE by maintaining a separate population for each hidden neuron position in the complete network. To our knowledge, the results of ESP are the best achieved so far in this task.

On the double pole balancing without velocity problem (DPNV), NEAT is compared to the only two systems that have been demonstrated able to solve the task: CE and ESP. The success of CE was first attributed to its ability to evolve structures. However, ESP, a fixed-topology NE system, was able to complete the task five times faster simply by restarting with a random number of hidden nodes whenever it got stuck. Our experiments will attempt to show that evolution of structure can lead to better performance if done right.

4.3.2 Double Pole Balancing with Velocities

The criteria for success on this task was keeping both poles balanced for 100,000 time steps (30 minutes of simulated time). A pole was considered balanced between -36 and 36 degrees from vertical. Fitness on this task was measured as the number of time steps that both poles remained balanced.

Table 1 shows that NEAT takes the fewest evaluations to complete this task, although the difference between NEAT and ESP is not statistically significant. The fixed-topology NE systems evolved networks with 10 hidden nodes, while NEAT's solutions always used between 0 and 4 hidden nodes. Thus, it is clear that NEAT's minimization

Table 1: Double pole balancing with velocity information (DPV). Evolutionary programming results were obtained by Saravanan and Fogel (1995). Conventional neuroevolution data was reported by Wieland (1991). SANE and ESP results were reported by Gomez and Miikkulainen (1999). In addition, Gruau et al. (1996) reported 34,000 evaluations in this task; however, their results are not directly comparable because they used a different fitness function (Equations 3 and 4). NEAT results are averaged over 120 experiments. All other results are averages over 50 runs. The standard deviation for the NEAT evaluations is 2,704 evaluations. Although standard deviations for other methods were not reported, if we assume similar variances, all differences are statistically significant ($p < 0.001$), except that between NEAT and ESP.

Method	Evaluations	Generations	No. Nets
Ev. Programming	307,200	150	2048
Conventional NE	80,000	800	100
SANE	12,600	63	200
ESP	3,800	19	200
NEAT	3,600	24	150

of dimensionality is working on this problem. The result is important because it shows that NEAT performs as well as ESP while finding more minimal solutions.

4.3.3 Double Pole Balancing Without Velocities

Gruau et al. (1996) introduced a special fitness function for this problem to prevent the system from solving the task simply by moving the cart back and forth quickly to keep the poles wiggling in the air. (Such a solution would not require computing the missing velocities.) Because both CE and ESP were evaluated using this special fitness function, NEAT uses it on this task as well. The fitness penalizes oscillations. It is the sum of two fitness component functions, f_1 and f_2 , such that $F = 0.1f_1 + 0.9f_2$. The two functions are defined over 1000 time steps:

$$f_1 = t/1000, \quad (3)$$

$$f_2 = \begin{cases} 0 & \text{if } t < 100, \\ \frac{0.75}{\sum_{i=t-100}^t (|x^i| + |\dot{x}^i| + |\theta_1^i| + |\dot{\theta}_1^i|)} & \text{otherwise.} \end{cases} \quad (4)$$

where t is the number of time steps the poles remain balanced during the 1,000 total time steps. The denominator in (4) represents the sum of offsets from center rest of the cart and the long pole. It is computed by summing the absolute value of the state variables representing the cart and long pole positions and velocities. Thus, by minimizing these offsets (damping oscillations), the system can maximize fitness. Because of this fitness function, swinging the poles is penalized, forcing the system to internally compute the hidden state variables.

Under Gruau et al.'s criteria for a solution, the champion of each generation is tested on generalization to make sure it is robust. This test takes a lot more time than the fitness test, which is why it is applied only to the champion. In addition to balancing both poles for 100,000 time steps, the winning controller must balance both poles from 625 different initial states, each for 1,000 times steps. The number of successes is called the *generalization performance of the solution*. In order to count as a solution, a network needs to generalize to at least 200 of the 625 initial states. Each start state is chosen by

Table 2: Double pole balancing without velocity information (DPNV). CE is Cellular Encoding of Gruau et al. (1996). ESP is Enforced Subpopulations of Gomez and Miikkulainen (1999). All results are averages over 20 simulations. The standard deviation for NEAT is 21,790 evaluations. Assuming similar variances for CE and ESP, all differences in number of evaluations are significant ($p < 0.001$). The generalization results are out of 625 cases in each simulation, and are not significantly different.

Method	Evaluations	Generalization	No. Nets
CE	840,000	300	16,384
ESP	169,466	289	1,000
NEAT	33,184	286	1,000

giving each state variable (i.e., x , $\dot{x}$, θ_1 , and $\dot{\theta}_1$) each of the values 0.05, 0.25, 0.5, 0.75, 0.95 scaled to the range of the input variable ($5^4 = 625$). At each generation, NEAT performs the generalization test on the champion of the highest-performing species that improved since the last generation.

Table 2 shows that NEAT is the fastest system on this challenging task. NEAT takes 25 times fewer evaluations than Gruau’s original benchmark, showing that the way in which structure is evolved has significant impact on performance. NEAT is also 5 times faster than ESP, showing that structure evolution can indeed perform better than evolution of fixed topologies. There was no significant difference in the ability of any of the 3 methods to generalize.

Why is NEAT so much faster than ESP on the more difficult task when there was not much difference in the easier task? The reason is that in the task without velocities, ESP needed to restart an average of 4.06 times per solution while NEAT never needed to restart. If restarts are factored out, the systems perform at similar rates. The best characterization of the difference is that NEAT is more reliable at avoiding deception. NEAT evolves many different structures simultaneously in different species, each representing a space of different dimensionality. Thus, NEAT is always trying many different ways to solve the problem at once, so it is less likely to get stuck.

The experimental results demonstrate both that NEAT can evolve structure when necessary, and that NEAT gains a significant performance advantage from doing so. We now turn to understanding how the system works, and whether it indeed solves the three problems with evolving a population of diverse topologies raised in the introduction.

5 Analysis of NEAT

We have argued that NEAT’s performance is due to historical markings, speciation, and incremental growth from minimal structure. In order to verify the contribution of each component, we performed a series of ablations. In addition, we introduce a new species visualization technique in order to better understand the dynamics of the system.

Ablations are meant to establish that each component of NEAT is necessary for its performance. For example, it is possible that growth from minimal structure is not really important; maybe the rest of the system, speciation and historical markings, is sufficient for NEAT’s optimal performance. This hypothesis will be checked by ablating both growth and starting from minimal structure from the system. On the other

Table 3: NEAT ablations summary. The table compares the average number of evaluations for a solution in the double pole balancing with velocities task. Each ablation leads to a weaker algorithm, showing that each component is necessary.

Method	Evaluations	Failure Rate
No-Growth NEAT (Fixed-Topologies)	30,239	80%
Nonspeciated NEAT	25,600	25%
Initial Random NEAT	23,033	5%
Nonmating NEAT	5,557	0
Full NEAT	3,600	0

hand, perhaps the situation is the opposite, and speciation buys nothing: protecting innovation might not be as important as we have argued. This hypothesis will be checked by ablating speciation from the system. Finally, we claimed that NEAT is able to make use of crossover even though genomes in NEAT have different sizes. This point is more controversial than it might seem. For example, Angeline et al. (1993) claimed that crossover in TWEANNs does more harm than good. We will check this hypothesis by ablating crossover from the system.

The reason why we do not ablate historical markings directly is that without historical markings the system would be a conventional NE system. Historical markings are the basis of every function in NEAT: Speciation uses a compatibility operator that is based on historical markings, and crossover would not be possible without them. All other system components can be ablated systematically.

5.1 Ablations Setup

Ablations can have a significant detrimental effect on performance, potentially to the point where the system cannot solve the task at all. Therefore, we used double pole balancing with velocities as the task for ablation studies. The task is complex enough to be interesting, yet still not too hard, so that ablated systems work as well. Thus, it is possible to compare the ablated versions of the system to the unablated system.

All settings were the same as in the double pole balancing with velocities experiment. Results are averages over 20 runs, except nonmating and full NEAT, which are averages over 120 runs (nonmating NEAT was fast enough to allow many runs).

5.2 Ablations Results

Table 3 shows the results of all the ablations, in terms of average evaluations required to find a solution. Averages in this table exclude trials that failed to find a solution in 1,000 generations. A failure rate denotes how often such failures occurred for each ablation. The main result is that the system performs significantly worse ($p \leq 0.001$) for every ablation. We will explain how each ablation was performed and then interpret the results.

5.3 No-Growth Ablation

Since populations in NEAT start out with no hidden nodes, simply removing growth from the system would disable NEAT by barring hidden nodes from all networks. NE systems where structures are fixed start with a fully connected hidden layer of neurons (Wieland, 1991). Therefore, to make the experiment fair, the no-growth ablation was also allowed to start with a fully connected hidden layer. Every genome specified 10

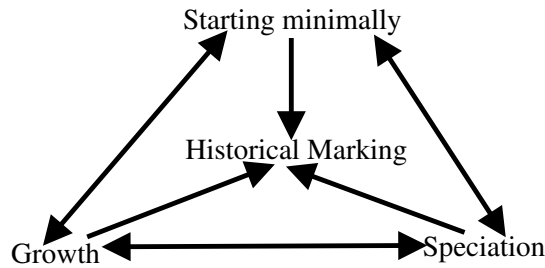


Figure 6: Dependencies among NEAT components. Strong interdependencies can be identified among the different components of NEAT.

hidden units like the fixed topology methods in this task (Saravanan and Fogel, 1995; Wieland, 1991). Without growth, NEAT was still able to speciate, but only based on weight differences. Given 1,000 generations to find a solution, the ablated system could only find a solution 20% of the time! When it did find a solution, it took 8.5 times more evaluations than full NEAT. Clearly, speciation and historical markings alone do not account for full NEAT's performance.

5.4 Initial Random Ablation

TWEANNs other than NEAT typically start with a random population (Angeline et al., 1993; Gruau et al., 1996; Yao, 1999). The structures in these systems can still grow (in most cases up to some bound). It is still possible that although growth is necessary, starting minimally is not.

We examined this question by starting evolution with random topologies as in other TWEANNs. Each network in the initial population received between 1 and 10 hidden neurons with random connectivity (as implemented by Pujol and Poli (1997)). The result is that random-starting NEAT was 7 times slower than full NEAT on average. The random-starting system also failed to find a solution within 1000 generations 5% of the time. The result suggests that starting randomly forces NE to search higher-dimensional spaces than necessary, thereby wasting time. If topologies are to grow, they should start out as small as possible.

5.5 Nonspeciated Ablation

We have argued that speciation is important because it protects innovation and allows search to proceed in many different spaces simultaneously. To test this claim, speciation should be ablated from the system. However, if this is done and nothing else is changed in the system, no structural innovations can survive, causing all networks to be stuck in minimal form.

To make the speciation ablation more meaningful, the nonspeciated NEAT must be started with an initial random population. This way, a variety of structures exist in the population from the beginning, and speciation is not necessary to attain structural diversity. The resulting nonspeciated NEAT was able to find solutions, although it failed in 25% of the attempts. When it found a solution, it was 7 times slower on average than full NEAT.

The reason for the dramatic slowdown is that without speciation, the population quickly converges on whatever topology happens to initially perform best. Thus, a lot of diversity is drained immediately (within 10 generations). On average, this initially

best-performing topology has about 5 hidden nodes. Thus, the population tends to converge to a relatively high-dimensional search space, even though the smaller networks in the initial population would have optimized faster. The smaller networks just do not get a chance because being small offers no immediate advantage in the initially random weight space. Of course, once in a while small networks are found that perform well, allowing a solution to be found more quickly. Whether or not such networks are found early on accounts for the large standard deviation of 41,704 evaluations.

The result shows that growth without speciation is not sufficient to account for NEAT's performance. For growth to succeed, it requires speciation, because speciation gives different structures a chance to optimize in their own niches.

5.6 Nonmating NEAT

For the last ablation, we removed mating from NEAT. This ablation tests the claim that crossover is a useful technique in TWEANNs. If NEAT's method of crossover works, then NEAT should perform significantly better with mating and mutation than with mutation alone.

A total of 120 simulations were run with crossover disabled but all other settings the same as before. It took on average 5,557 evaluations to find a solution without mating, compared to 3,600 with mating enabled. The difference is statistically significant ($p = 0.001$). Thus, it is clear that mating *does* contribute when it is done right. However, the nonmating version of NEAT is still significantly faster than the other ablations.

5.7 Ablation Conclusions

An important conclusion is that all of the parts of NEAT work together (Figure 6). None of the system can work without historical markings because all of NEAT's functions utilize historical markings. If growth from minimal structure is removed, speciation can no longer help NEAT find spaces with minimal dimensionality. If speciation is removed, growth from minimal structures cannot proceed because structural innovations do not survive. When the system starts with a population of random topologies without speciation, the system quickly converges onto a nonminimal topology that just happens to be one of the best networks in the initial population. Thus, each component is necessary to make NEAT work.

5.8 Visualizing Speciation

The ablation studies demonstrate that speciation is a necessary part of the overall system. To understand how innovation takes place in NEAT, it is important to understand the dynamics of speciation. How many species form over the course of a run? How often do new species arise? How often do species die? How large do the species get? We answer these questions by depicting speciation visually over time.

Figure 7 depicts a typical run of the double pole balancing with velocities task. In this run, the task took 29 generations to complete, which is slightly above average. In the visualization, successive generations are shown from top to bottom. Species are depicted horizontally for each generation, with the width of each species proportional to its size during the corresponding generation. Species are divided from each other by white lines, and new species always arrive on the right hand side. A species becomes bright when the fitness of its most fit member is one standard deviation above the mean fitness of the run, indicating that the species is a highly promising one. A species becomes very bright when it is two standard deviations above the mean, suggesting that the species is very close to a solution. Thus, it is possible to follow any species

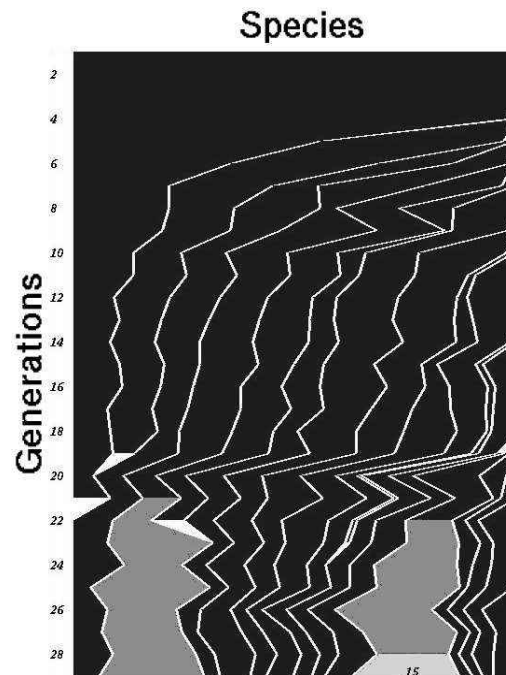


Figure 7: Visualizing speciation during a run of the double pole balancing with velocity information task. Two species begin to close in on a solution soon after the 20th generation. Around the same time, some of the oldest species become extinct.

from its inception to the end of the run.

Figure 7 shows that the initial minimal topology species was the only species in the population until the 5th generation. Recall that species are computed by the system according to a compatibility distance metric, indicating that before generation 7, all organisms were sufficiently compatible to be grouped into a single species. The visualization shows how the initial species shrinks dramatically in order to make room for the new species.

A few species can be observed becoming extinct during this run. When a species becomes extinct, we see a white triangle between the generation it expired and the next generation. Thus, in this run, the initial species finally became extinct at the 21st generation after shrinking for a long time. It was unable to compete with newer, more innovative species. The second species to appear in the population met a similar fate in the 19th generation.

In the 21st generation, a structural mutation in the second-oldest surviving species connected the long pole angle sensor to a hidden node that had previously only been connected to the cart position sensor. This gave networks in the species the new capability to combine these observations, leading to a significant boost in fitness (and brightening of the species in Figure 7). The innovative species subsequently expanded but did not take over the population. Nearly simultaneously, in the 22nd generation, a younger species also made its own useful connection, this time between the short pole velocity sensor and long pole angle sensor, leading to its own subsequent expansion. In the 28th generation, this same species made a pivotal connection between the cart

position and its already established method for comparing short pole velocity to long pole angle. This innovation was enough to solve the problem within one generation of additional weight mutations. In the final generation, the winning species was 11 generations old and included 38 neural networks out of the population of 150.

Most of the species that did not come close to a solution survived the run even though they fell significantly behind around the 21st generation. This observation is important, because it visually demonstrates that innovation is indeed being protected. The winning species does not take over the entire population.

Ablation studies confirm the interdependence of all of NEAT's components, and the speciation visualization offers a means of visualizing the dynamics of the system. We now turn to a discussion of the advantages and shortcomings of the method and its future potential.

6 Discussion and Future Work

NEAT presents several advances in the evolution of neural networks. Through historical markings, NEAT offers a solution to the problem of competing conventions in a population of diverse topologies. NEAT also demonstrates that a meaningful metric for comparing and clustering similar networks easily derives from the availability of historical information in the population, such that costly topological analysis is not necessary to speciate or mate networks. Our ablation studies confirm the hypothesis that starting with a population of minimal topologies is advantageous. Finally, performance comparisons suggest that the evolution of structure can be used to gain efficiency over the evolution of fixed topologies.

A parallel can be drawn between structure evolution in NEAT and incremental evolution (Gomez and Miikkulainen, 1997; Wieland, 1991). Incremental evolution is a method used to train a system to solve harder tasks than it normally could by training it on incrementally more challenging tasks. NE is likely to get stuck on a local optimum when attempting to solve the harder task directly. However, after solving the easier version of the task first, the population is likely to be in a part of fitness space closer to the solution to the harder task, allowing it to avoid local optima. Adding structure to a solution is analogous to taking a solution to an easy task as the starting point for evolving the solution to a harder task. The network structure before the addition is optimized in a lower-dimensional space. When structure is added, the network increments into a more complex space where it is already close to the solution. The difference between the incrementality of adding structure and general incremental evolution is that adding structure is *automatic* in NEAT, whereas a sequence of progressively harder tasks requires human design.

A key insight behind NEAT is that it is *not* the ultimate structure of the solution that really matters, but rather the structure of all the intermediate solutions along the way to finding the solution. The connectivity of every intermediate solution represents a parameter space that evolution must optimize, and the more connections there are, the more parameters need to be optimized. Therefore, if the amount of structure can be minimized throughout evolution, so can the dimensionality of the spaces being explored, leading to significant performance gains.

Figure 8 illustrates the advantage of evolving minimal structures with a picture of an elegant solution to the DPNV task. All evolved solutions displayed the same pattern of weights on direct connections from the input nodes to the output nodes. The connections from nodes one and two were always inhibitory, while those from nodes three and four were always excitatory. NEAT also evolved a recurrent loop on the single

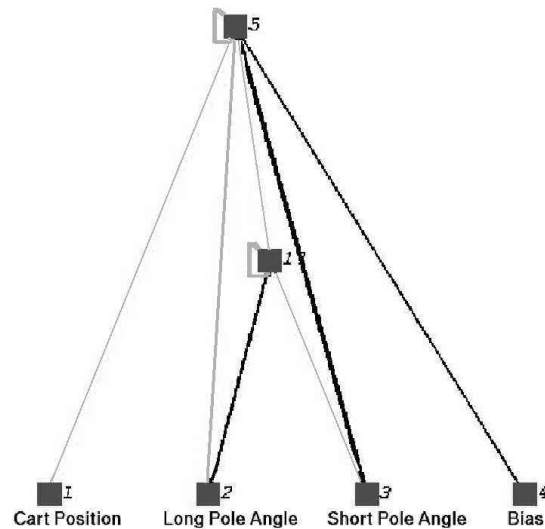


Figure 8: A NEAT solution to the DPNV problem. This clever solution works by taking the derivative of the difference in pole angles. Using the recurrent connection to itself, the single hidden node determines whether the poles are falling away or towards each other. This solution allows controlling the system without computing the velocities of each pole separately. Without evolving structure, it would be difficult to discover such subtle and compact solutions. Starting minimally makes discovering such compact solutions more likely.

output node of all solution networks. In 90% of solutions, this recurrent connection was excitatory and served to maintain continuity in the actions. Upon this general theme, solutions included varying degrees of structural elaborations utilizing from zero to two hidden nodes. For example, the solution shown in Figure 8 uses one hidden node to determine whether the poles are moving towards or away from one another. The basic underlying structure is the same across all these solutions and suggests that once a proper minimal foundation is discovered, a variety of paths open up to a solution.

In order to minimize structure throughout evolution, NEAT incrementally elaborates structure in a stochastic manner from a minimal starting point. Because of speciation, useful elaborations survive even if they are initially detrimental. Thus, NEAT strengthens the analogy between GAs and natural evolution by not only performing the optimizing function of evolution, but also a *complexifying* function, allowing solutions to become incrementally more complex at the same time as they become more optimal.

It is this complexifying function that makes NEAT unique among GAs. Although GAs have been proposed where bit string chromosomes can increase in length indefinitely (Harvey, 1993), NEAT goes beyond a gradual uniform growth in genome size. While Harvey's method increases the size of a chromosome incrementally across the entire population, NEAT simultaneously searches over *different* landscapes, all complexifying in different ways.

Such speciated search of incrementally increasing complexity offers a possibly powerful new approach to the problem of competitive coevolution. In competitive coevolution, increasingly sophisticated strategies are evolved by allowing networks in a

population to compete against each other. The hope is that an “arms race” will force the opponents to continually evolve strategies better than the strategies of other networks in the population. This method is useful because it can produce high-level strategies and tactics without the need for an expert player to teach the system. Ideally, strategies should become more sophisticated as evolution progresses. However, evolution tends to find the simplest solutions that can win, meaning that strategies oscillate between different idiosyncratic yet uninteresting variations (Darwen, 1996; Rosin and Belew, 1997).

We hypothesize that once competitive coevolution converges onto a dominant strategy, it cannot be improved upon, because it takes the entire set of connection weight values to represent the strategy. Altering the weights means altering the strategy, rather than building upon and complexifying the strategy. Thus, if a new strategy is to take hold, it must win by being *different* than the previous dominant strategy, rather than by being *more sophisticated*. In contrast, NEAT can evolve increasingly more sophisticated strategies continually, because as soon as the population converges on a new dominant strategy, new connections and new nodes can be added to the current strategy. New structure means new *expressive space* for *elaborating* on the existing strategy, rather than replacing it. Thus, this approach allows *continual coevolution*, i.e., non-convergent innovation on dominant strategies. In addition, because different strategies in different species are protected, there will be multiple dominant and continually more complex strategies.

In addition to continual coevolution, the evolution of structure should allow the integration of separate expert neural networks. For example, suppose one neural network can “kick” a ball towards the goal from any position on a field, and another network can dribble the ball around the field without losing control. Neither of these two networks alone can play soccer. However, if we could somehow combine their expertise into one, perhaps we could get a soccer player out. Combining these controllers is not a simple matter of processing both their outputs. Just because a robot can dribble and shoot does not mean it knows *where* to dribble or *when* to shoot. Both shooting and dribbling affect each other as well. Where you dribble affects how easy your shot is, and shooting forces a robot to stop dribbling. In order to optimally combine the two skills, the hidden nodes of the two networks must share information so that the new combined expert can make intelligent decisions and combine the two skills effectively. We hypothesize that NEAT has the capability of searching for the right interconnections between two distinct networks to create an integrated supernetwork that takes advantage of the expertise of both its component networks.

Finally, we would like to establish a characterization of what NEAT is best suited for. The experimental results show the difference between ESP and NEAT is significantly higher on the hardest pole balancing task. This result implies that evolving diverse topologies is particularly suited for problems where other methods are likely to get stuck. Such problems may be deceptive, meaning local optima have large basins of attraction compared to global optima, and the global optima are significantly different from the local optima (Goldberg, 1989). Because NEAT can always add more structure, it is not necessarily trapped even if the current weights of a networks represent a local optimum in fitness space. By adding additional structure, NEAT adds new dimensions to weight space, thereby opening up potential new avenues for escape. We plan to test NEAT on problems with varying fitness landscapes to get a better idea of the kinds of problems the method tackles best.

7 Conclusion

The main conclusion is that NEAT is a powerful method for artificially evolving neural networks. NEAT demonstrates that evolving topology along with weights can be made a major advantage. Experimental comparisons verify that such evolution is several times more efficient than the neuroevolution methods so far. Ablation studies show that historical markings, protection of innovation through speciation, and incremental growth from minimal structure all work together to produce a system that is capable of evolving solutions of minimal complexity. NEAT strengthens the analogy between GAs and natural evolution by both optimizing *and complexifying* solutions simultaneously. We believe that the capacity to complexify solutions over the course of evolution offers the possibility of continual competitive coevolution and evolution of combinations of experts in the future.

Acknowledgments

This research was supported in part by the National Science Foundation under grant IIS-0083776 and by the Texas Higher Education Coordinating Board under grant ARP-003658-476-2001. Thanks to Faustino Gomez for providing pole balancing code and Michael Shao for pointers to the biology literature.

References

- Anderson, C. W. (1989). Learning to control an inverted pendulum using neural networks. *IEEE Control Systems Magazine*, 9:31–37.
- Angeline, P. J., Saunders, G. M., and Pollack, J. B. (1993). An evolutionary algorithm that constructs recurrent neural networks. *IEEE Transactions on Neural Networks*, 5:54–65.
- Branke, J. (1995). Evolutionary algorithms for neural network design and training. In Alander, J. T., editor, *Proceedings First Nordic Workshop on Genetic Algorithms and their Applications*, pages 145–163, University of Vaasa Press, Vaasa, Finland.
- Braun, H. and Weisbrod, J. (1993). Evolving feedforward neural networks. In Albrecht, R. F., Reeves, C. R., and Steele, N. C., editors, *Proceedings of ANNGA93, International Conference on Artificial Neural Networks and Genetic Algorithms*, pages 25–32, Springer-Verlag, Innsbruck.
- Chen, D. et al. (1993). Constructive learning of recurrent neural networks. In Petsche, T., Judd, S., and Hanson, S., editors, *Computational Learning Theory and Natural Learning Systems III*. MIT Press, Cambridge, Massachusetts.
- Cybenko, G. (1989). Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals, and Systems*, 2(4):303–314.
- Darnell, J. E. and Doolittle, W. F. (1986). Speculations on the early course of evolution. *Proceedings of the National Academy of Sciences, USA*, 83:1271–1275.
- Darwen, P. J. (1996). *Co-Evolutionary Learning by Automatic Modularisation with Speciation*. Ph.D. thesis, School of Computer Science, University College, University of New South Wales, Sydney, Australia.
- Darwen, P. and Yao, X. (1996). Automatic modularization by speciation. In *Proceedings of the 1996 IEEE International Conference on Evolutionary Computation (ICEC '96)*, pages 88–93, Nagoya, IEEE Press, Piscataway, New Jersey.
- Dasgupta, D. and McGregor, D. (1992). Designing application-specific neural networks using the structured genetic algorithm. In Whitley, D. and Schaffer, J. D., editors, *Proceedings of the International Conference on Combinations of Genetic Algorithms and Neural Networks*, pages 87–96, IEEE Press, Piscataway, New Jersey.

- Fullmer, B. and Miikkulainen, R. (1992). Using marker-based genetic encoding of neural networks to evolve finite-state behaviour. In Varela, F. J. and Bourgine, P., editors, *Toward a Practice of Autonomous Systems: Proceedings of the First European Conference on Artificial Life*, pages 255–262, MIT Press, Cambridge, Massachusetts.
- Goldberg, D. E. (1989). *Genetic Algorithms in Search, Optimization and Machine Learning*. Addison-Wesley, Reading, Massachusetts.
- Goldberg, D. E. and Richardson, J. (1987). Genetic algorithms with sharing for multimodal function optimization. In Grefenstette, J. J., editor, *Proceedings of the Second International Conference on Genetic Algorithms*, pages 148–154, Morgan Kaufmann, San Francisco, California.
- Gomez, F. and Miikkulainen, R. (1997). Incremental evolution of complex general behavior. *Adaptive Behavior*, 5:317–342.
- Gomez, F. and Miikkulainen, R. (1999). Solving non-Markovian control tasks with neuroevolution. In Dean, T., editor, *Proceedings of the Sixteenth International Joint Conference on Artificial Intelligence*, pages 1356–1361, Morgan Kaufmann, San Francisco, California.
- Gomez, F. and Miikkulainen, R. (2002). Learning robust nonlinear control with neuroevolution. Technical Report AI02-292, Department of Computer Sciences, The University of Texas at Austin, Austin, Texas.
- Gruau, F. (1993). Genetic synthesis of modular neural networks. In Forrest, S., editor, *Proceedings of the Fifth International Conference on Genetic Algorithms*, pages 318–325, Morgan Kaufmann, San Francisco, California.
- Gruau, F., Whitley, D., and Pyeatt, L. (1996). A comparison between cellular encoding and direct encoding for genetic neural networks. In Koza, J. R. et al., editors, *Genetic Programming 1996: Proceedings of the First Annual Conference*, pages 81–89, MIT Press, Cambridge, Massachusetts.
- Harvey, I. (1993). *The Artificial Evolution of Adaptive Behavior* Ph.D. thesis, School of Cognitive and Computing Sciences, University of Sussex, Sussex, UK.
- Holland, J. H. (1975). *Adaptation in Natural and Artificial Systems: An Introductory Analysis with Applications to Biology, Control and Artificial Intelligence*. University of Michigan Press, Ann Arbor, Michigan.
- Kaelbling, L. P., Littman, M., and Moore, A. W. (1996). Reinforcement learning: A survey. *Journal of Artificial Intelligence*, 4:237–285.
- Krishnan, R. and Ciesielski, V. B. (1994). Delta-gann: A new approach to training neural networks using genetic algorithms. In Tsoi, A. C. and Downs, T., editors, *Proceedings of the Australian Conference on Neural Networks*, pages 194–197, University of Queensland, Brisbane, Australia.
- Lee, C.-H. and Kim, J.-H. (1996). Evolutionary ordered neural network with a linked-list encoding scheme. In *Proceedings of the 1996 IEEE International Conference on Evolutionary Computation*, pages 665–669, IEEE Press, Piscataway, New Jersey.
- Mahfoud, S. W. (1995). *Niching Methods for Genetic Algorithms* Ph.D. thesis, Department of General Engineering, IlliGAL Report 95001, University of Illinois at Urbana-Champaign, Urbana, Illinois.
- Mandischer, M. (1993). Representation and evolution of neural networks. In Albrecht, R. F., Reeves, C. R., and Steele, U. C., editors, *Artificial Neural Nets and Genetic Algorithms*, pages 643–649, Springer Verlag, New York, New York.
- Maniezzo, V. (1994). Genetic evolution of the topology and weight distribution of neural networks. *IEEE Transactions on Neural Networks*, 5(1):39–53.

- Montana, D. J. and Davis, L. (1989). Training feedforward neural networks using genetic algorithms. In Sridharan, S., editor, *Proceedings of the Eleventh International Joint Conference on Artificial Intelligence*, pages 762–767, Morgan Kaufmann, San Francisco, California.
- Moriarty, D. E. (1997). *Symbiotic Evolution of Neural Networks in Sequential Decision Tasks*. Ph.D. thesis, Department of Computer Sciences, The University of Texas at Austin. Technical Report UT-AI97-257.
- Moriarty, D. E. and Miikkulainen, R. (1996). Efficient reinforcement learning through symbiotic evolution. *Machine Learning*, 22:11–32.
- Moriarty, D. E. and Miikkulainen, R. (1997). Forming neural networks through efficient and adaptive co-evolution *Evolutionary Computation*, 5(4):373–399.
- Opitz, D. W. and Shavlik, J. W. (1997). Connectionist theory refinement: Genetically searching the space of network topologies. *Journal of Artificial Intelligence Research*, 6:177–209.
- Pendrith, M. (1994). On reinforcement learning of control actions in noisy and non-Markovian domains. Technical Report UNSW-CSE-TR-9410, School of Computer Science and Engineering, The University of New South Wales, Sydney, Australia.
- Potter, M. A. and De Jong, K. A. (1995). Evolving neural networks with collaborative species. In Oren, T. I. and Birta, L. G., editors, *Proceedings of the 1995 Summer Computer Simulation Conference*, pages 340–345, Society for Computer Simulation, San Diego, California..
- Potter, M. A., De Jong, K. A., and Grefenstette, J. J. (1995). A coevolutionary approach to learning sequential decision rules. In Eshelman, L. J., editor, *Proceedings of the Sixth International Conference on Genetic Algorithms*, pages 366–372, Morgan Kaufmann, San Francisco, California.
- Pujol, J. C. F. and Poli, R. (1998). Evolving the topology and the weights of neural networks using a dual representation. *Special Issue on Evolutionary Learning of the Applied Intelligence Journal*, 8(1):73–84.
- Radcliffe, N. J. (1993). Genetic set recombination and its application to neural network topology optimisation. *Neural Computing and Applications*, 1(1):67–90.
- Radding, C. M. (1982). Homologous pairing and strand exchange in genetic recombination. *Annual Review of Genetics*, 16:405–437.
- Rosin, C. D. and Belew, R. K. (1997). New methods for competitive evolution. *Evolutionary Computation*, 5(1):1–29.
- Saravanan, N. and Fogel, D. B. (1995). Evolving neural control systems. *IEEE Expert*, 10(3):23–27.
- Schaffer, J. D., Whitley, D., and Eshelman, L. J. (1992). Combinations of genetic algorithms and neural networks: A survey of the state of the art. In Whitley, D. and Schaffer, J., editors, *Proceedings of the International Workshop on Combinations of Genetic Algorithms and Neural Networks (COGANN-92)*, pages 1–37, IEEE Press, Piscataway, New Jersey.
- Sigal, N. and Alberts, B. (1972). Genetic recombination: The nature of a crossed strand-exchange between two homologous DNA molecules. *Journal of Molecular Biology*, 71(3):789–793.
- Spears, W. (1995). Speciation using tag bits. In *Handbook of Evolutionary Computation*. IOP Publishing Ltd. and Oxford University Press, Oxford, UK.
- Thierens, D. (1996). Non-redundant genetic coding of neural networks. In *Proceedings of the IEEE International Conference on Evolutionary Computation*, pages 571–575, IEEE Press, Piscataway, New Jersey.
- Watson, J. D. et al. (1987). *Molecular Biology of the Gene Fourth Edition*. The Benjamin Cummings Publishing Company, Inc., Menlo Park, California.

- Whitley, D. et al. (1993). Genetic reinforcement learning for neurocontrol problems. *Machine Learning*, 13:259–284.
- Wieland, A. (1991). Evolving neural network controllers for unstable systems. In *Proceedings of the International Joint Conference on Neural Networks*, pages 667–673, IEEE Press, Piscataway, New Jersey.
- Yao, X. (1999). Evolving artificial neural networks. *Proceedings of the IEEE*, 87(9):1423–1447.
- Yao, X. and Liu, Y. (1996). Towards designing artificial neural networks by evolution. *Applied Mathematics and Computation*, 91(1):83–90.
- Yao, X. and Shi, Y. (1995). A preliminary study on designing artificial neural networks using co-evolution. In *Proceedings of the IEEE Singapore International Conference on Intelligent Control and Instrumentation*, pages 149–154, IEEE Singapore Section.
- Zhang, B.-T. and Mühlenbein, H. (1993). Evolving optimal neural networks using genetic algorithms with Occam's razor. *Complex Systems*, 7:199–220.